

ecsnet

Generated by Doxygen 1.14.0

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

component_descriptor_t	??
component_storage_t	
Storage container for a single component type. Each registered component type gets its own storage structure. It contains:	??
connection_manager_t	
Manages all active network connections (peers) for a client or server	??
dirty_component_t	
Dirty flag used by the networking layer to sync ECS data. It marks whether an entity's component has been modified	??
ecs_t	
Internal structure representing the ECS world state. This holds:	??
entity_meta_t	
Metadata for an entity. This structure holds internal bookkeeping information for an entity, such as whether it is currently active (in use) in the ECS world	??
net_socket_t	
Represents a network socket. This struct encapsulates the socket's file descriptor, type, and address information	??
network_architecture_config_t	
Structure for configuring the network architecture	??
network_architecture_t	
Represents the opaque network architecture structure. This struct serves as a wrapper that holds a pointer to the specific implementation (e.g., Client-Server, P2P) and a reference to the ECS	??
network_cs_t	
Represents the client-server network architecture implementation. This struct contains the core components for a client-server setup, including the connection manager, protocol handler, and a pointer to the ECS instance	??
network_id_pair_t	
Pair mapping between a global network ID and a local entity ID	??
network_map_t	
Dynamic mapping structure from network IDs to local entity IDs. This map grows dynamically as new network IDs are encountered. Clients rely on this to map persistent server-assigned IDs to their local ECS entities	??
network_packet_t	
A full network packet structure. This struct combines the header and the data payload, providing a complete packet representation for sending and receiving	??

networked_entity_t	Component representing network identity and interest groups	??
packet_header_t	The header of a network packet. Contains metadata about the packet, such as its size and type. Note on padding: The size is 2 bytes and the enum type is 4 bytes, which results in 2 bytes of padding to align the struct to an 8-byte boundary	??
peer_t	Represents a connected network peer	??
position_t	Component data structure for entity position	??
protocol_handler_t	The protocol handler structure. Manages the state for incoming and outgoing network packets, providing functionality for packing, unpacking, and processing network data	??
rotation_t	Component data structure for entity rotation (quaternion format)	??
transform_t	Component data structure for entity transformation. Combines position and rotation into a single component	??
velocity_t	Component data structure for entity velocity	??

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

include/config.h	??
include/connection_manager.h	??
include/ecs.h	??
include/ecs_builtin.h	??
include/ecs_internal.h	??
include/ecs_types.h	??
include/ecsnet.h	??
include/net_socket.h	??
include/network_architecture.h	??
include/network_cs.h	??
include/network_map.h	??
include/networked_entity.h	??
Defines the NetworkedEntity component used for network replication	??
include/protocol_handler.h	??

Chapter 3

Class Documentation

3.1 component_descriptor_t Struct Reference

Public Attributes

- size_t [size](#)
- const char * [name](#)
- serialize_func_t [serialize](#)
- deserialize_func_t [deserialize](#)

3.1.1 Member Data Documentation

3.1.1.1 deserialize

```
deserialize_func_t component_descriptor_t::deserialize
```

Pointer to the deserialization function.

3.1.1.2 name

```
const char* component_descriptor_t::name
```

Human-readable component name.

3.1.1.3 serialize

```
serialize_func_t component_descriptor_t::serialize
```

Pointer to the serialization function.

3.1.1.4 size

```
size_t component_descriptor_t::size
```

Size of the component in bytes.

The documentation for this struct was generated from the following file:

- include/ecs_types.h

3.2 component_storage_t Struct Reference

Storage container for a single component type. Each registered component type gets its own storage structure. It contains:

```
#include <ecs_internal.h>
```

Collaboration diagram for component_storage_t:

Public Attributes

- [component_descriptor_t](#) descriptor
- void * [data](#)
- bool * [used](#)
- bool * [is_dirty](#)
- size_t [capacity](#)

3.2.1 Detailed Description

Storage container for a single component type. Each registered component type gets its own storage structure. It contains:

- The descriptor describing the component type (size, name, serialization functions).
- A contiguous block of memory for all entities' data of this component type.
- Arrays to track which entities have this component ([used](#)) and which ones have been modified ([is_dirty](#)).

3.2.2 Member Data Documentation

3.2.2.1 capacity

```
size_t component_storage_t::capacity
```

Current capacity for entity slots (matches `ecs->entity_capacity`).

3.2.2.2 data

```
void* component_storage_t::data
```

Contiguous memory storing component data for all entities.

3.2.2.3 descriptor

```
component_descriptor_t component_storage_t::descriptor
```

Metadata describing the component type.

3.2.2.4 is_dirty

```
bool* component_storage_t::is_dirty
```

Dynamic array indicating which components have been modified.

3.2.2.5 used

```
bool* component_storage_t::used
```

Dynamic array indicating which entities have this component.

The documentation for this struct was generated from the following file:

- include/ecs_internal.h

3.3 connection_manager_t Struct Reference

Manages all active network connections (peers) for a client or server.

```
#include <connection_manager.h>
```

Collaboration diagram for connection_manager_t:

Public Attributes

- [peer_t](#) [peers](#) [MAX_PEERS]
- int [peer_count](#)
- [net_socket_t](#) [listen_sockets](#) [PROTOCOL_COUNT]
- bool [is_server](#)
- void * [user_data](#)
- void(* [on_receive](#))(void *, [peer_t](#) *, const void *, int)
- void(* [on_connect](#))(void *, [peer_t](#) *)
- void(* [on_disconnect](#))(void *, [peer_t](#) *)

3.3.1 Detailed Description

Manages all active network connections (peers) for a client or server.

This structure holds an array of all connected peers, manages the listen sockets, and stores pointers to peer related callback functions for network events.

3.3.2 Member Data Documentation

3.3.2.1 is_server

```
bool connection_manager_t::is_server
```

A flag indicating if this instance is a server.

3.3.2.2 listen_sockets

```
net_socket_t connection_manager_t::listen_sockets[PROTOCOL_COUNT]
```

Sockets for listening on both TCP and UDP.

3.3.2.3 on_connect

```
void(* connection_manager_t::on_connect) (void *, peer_t *)
```

The callback for when a new peer connects.

3.3.2.4 on_disconnect

```
void(* connection_manager_t::on_disconnect) (void *, peer_t *)
```

The callback for when a peer disconnects.

3.3.2.5 on_receive

```
void(* connection_manager_t::on_receive) (void *, peer_t *, const void *, int)
```

The callback for when data is received.

3.3.2.6 peer_count

```
int connection_manager_t::peer_count
```

The current number of active peers.

3.3.2.7 peers

```
peer_t connection_manager_t::peers[MAX_PEERS]
```

An array of all managed peers.

3.3.2.8 user_data

```
void* connection_manager_t::user_data
```

User data passed to callbacks

The documentation for this struct was generated from the following file:

- include/connection_manager.h

3.4 dirty_component_t Struct Reference

Dirty flag used by the networking layer to sync ECS data. It marks whether an entity's component has been modified.

```
#include <ecs.h>
```

Public Attributes

- component_t **component_id**
- const void * **data**

3.4.1 Detailed Description

Dirty flag used by the networking layer to sync ECS data. It marks whether an entity's component has been modified.

The documentation for this struct was generated from the following file:

- include/ecs.h

3.5 ecs_t Struct Reference

Internal structure representing the ECS world state. This holds:

```
#include <ecs_internal.h>
```

Collaboration diagram for ecs_t:

Public Attributes

- [entity_meta_t](#) * [entities](#)
- [component_signature_t](#) * [signatures](#)
- [size_t](#) [entity_capacity](#)
- [component_storage_t](#) * [components](#)
- [size_t](#) [component_capacity](#)
- [uint32_t](#) [registered_entities_count](#)
- [uint32_t](#) [registered_component_count](#)
- [system_func_t](#) [systems](#) [MAX_SYSTEMS]
- [int](#) [system_count](#)

3.5.1 Detailed Description

Internal structure representing the ECS world state. This holds:

- All entity metadata.
- Component storage for every registered component type.
- A list of registered systems and the total count.
- The number of registered components.

This is the core ECS structure passed to all internal and public ECS functions.

3.5.2 Member Data Documentation

3.5.2.1 [component_capacity](#)

```
size_t ecs_t::component_capacity
```

Current capacity of the component array.

3.5.2.2 [components](#)

```
component_storage_t* ecs_t::components
```

Dynamic array of registered component storages.

3.5.2.3 [entities](#)

```
entity_meta_t* ecs_t::entities
```

Dynamic array of entity metadata.

3.5.2.4 entity_capacity

```
size_t ecs_t::entity_capacity
```

Current capacity of the entity arrays.

3.5.2.5 registered_component_count

```
uint32_t ecs_t::registered_component_count
```

Total number of registered component types.

3.5.2.6 registered_entities_count

```
uint32_t ecs_t::registered_entities_count
```

Total number of registered entities.

3.5.2.7 signatures

```
component_signature_t* ecs_t::signatures
```

Dynamic array of component signatures per entity.

3.5.2.8 system_count

```
int ecs_t::system_count
```

Current number of registered systems.

3.5.2.9 systems

```
system_func_t ecs_t::systems[MAX_SYSTEMS]
```

Array of registered system function pointers.

The documentation for this struct was generated from the following file:

- include/ecs_internal.h

3.6 entity_meta_t Struct Reference

Metadata for an entity. This structure holds internal bookkeeping information for an entity, such as whether it is currently active (in use) in the ECS world.

```
#include <ecs_internal.h>
```

Public Attributes

- bool [in_use](#)

3.6.1 Detailed Description

Metadata for an entity. This structure holds internal bookkeeping information for an entity, such as whether it is currently active (in use) in the ECS world.

3.6.2 Member Data Documentation

3.6.2.1 in_use

```
bool entity_meta_t::in_use
```

Indicates if this entity slot is currently allocated.

The documentation for this struct was generated from the following file:

- include/ecs_internal.h

3.7 net_socket_t Struct Reference

Represents a network socket. This struct encapsulates the socket's file descriptor, type, and address information.

```
#include <net_socket.h>
```

Public Attributes

- struct sockaddr_in [addr](#)
- socket_type_t [type](#)
- int [fd](#)

3.7.1 Detailed Description

Represents a network socket. This struct encapsulates the socket's file descriptor, type, and address information.

3.7.2 Member Data Documentation

3.7.2.1 addr

```
struct sockaddr_in net_socket_t::addr
```

The address associated with the socket.

3.7.2.2 fd

```
int net_socket_t::fd
```

The socket file descriptor.

3.7.2.3 type

```
socket_type_t net_socket_t::type
```

The type of the socket (TCP or UDP).

The documentation for this struct was generated from the following file:

- include/net_socket.h

3.8 network_architecture_config_t Struct Reference

Structure for configuring the network architecture.

```
#include <network_architecture.h>
```

Public Attributes

- network_architecture_type_t [type](#)
- const char * [ip_address](#)
- uint16_t [port](#)
- bool [is_server](#)
- uint16_t [tcp_port](#)
- uint16_t [udp_port](#)
- float [ecs_sync_hz](#)
- void(* [on_peer_connected](#))(void *[user_data](#), [peer_t](#) *peer)
- void(* [on_peer_disconnected](#))(void *[user_data](#), [peer_t](#) *peer)
- void(* [on_packet_received](#))(void *[user_data](#), [peer_t](#) *peer, const void *data, int len)
- void(* [on_client_input](#))(void *[user_data](#), [peer_t](#) *from, entity_t entity_id, uint8_t cmd, const void *extra, uint16_t extra_len)
- void * [user_data](#)

3.8.1 Detailed Description

Structure for configuring the network architecture.

3.8.2 Member Data Documentation

3.8.2.1 ecs_sync_hz

```
float network_architecture_config_t::ecs_sync_hz
```

The ECS synchronization frequency in Hz.

3.8.2.2 ip_address

```
const char* network_architecture_config_t::ip_address
```

The IP address to bind to or connect to.

3.8.2.3 is_server

```
bool network_architecture_config_t::is_server
```

A flag indicating if the instance is a server.

3.8.2.4 port

```
uint16_t network_architecture_config_t::port
```

The main port for communication.

3.8.2.5 tcp_port

```
uint16_t network_architecture_config_t::tcp_port
```

The TCP port to use.

3.8.2.6 type

```
network_architecture_type_t network_architecture_config_t::type
```

The type of network architecture to use.

3.8.2.7 udp_port

```
uint16_t network_architecture_config_t::udp_port
```

The UDP port to use.

3.8.2.8 user_data

```
void* network_architecture_config_t::user_data
```

User data passed to callbacks

The documentation for this struct was generated from the following file:

- include/network_architecture.h

3.9 network_architecture_t Struct Reference

Represents the opaque network architecture structure. This struct serves as a wrapper that holds a pointer to the specific implementation (e.g., Client-Server, P2P) and a reference to the ECS.

```
#include <network_architecture.h>
```

Collaboration diagram for network_architecture_t:

Public Attributes

- network_architecture_type_t **type**
- [network_architecture_config_t](#) **config**
- void * **impl**
- [ecs_t](#) * **ecs**

3.9.1 Detailed Description

Represents the opaque network architecture structure. This struct serves as a wrapper that holds a pointer to the specific implementation (e.g., Client-Server, P2P) and a reference to the ECS.

The documentation for this struct was generated from the following file:

- include/network_architecture.h

3.10 network_cs_t Struct Reference

Represents the client-server network architecture implementation. This struct contains the core components for a client-server setup, including the connection manager, protocol handler, and a pointer to the ECS instance.

```
#include <network_cs.h>
```

Collaboration diagram for network_cs_t:

Public Attributes

- [ecs_t](#) * **ecs**
- [network_architecture_config_t](#) **config**
- [connection_manager_t](#) **connection_manager**
- [protocol_handler_t](#) **protocol_handler**
- float **sync_acc**
- [network_map_t](#) **network_map**
- uint32_t **next_network_id**
- uint32_t * **pending_destroy_ids**
Pending destroy list for server-side entity removal.
- size_t **pending_destroy_count**
- size_t **pending_destroy_capacity**

3.10.1 Detailed Description

Represents the client-server network architecture implementation. This struct contains the core components for a client-server setup, including the connection manager, protocol handler, and a pointer to the ECS instance.

3.10.2 Member Data Documentation

3.10.2.1 config

```
network_architecture_config_t network_cs_t::config
```

Configuration for this network instance.

3.10.2.2 connection_manager

```
connection_manager_t network_cs_t::connection_manager
```

The connection manager instance.

3.10.2.3 ecs

```
ecs_t* network_cs_t::ecs
```

A pointer to the ECS instance.

3.10.2.4 network_map

```
network_map_t network_cs_t::network_map
```

Mapping of remote network IDs to local entity IDs on the client. Initialized on both client and server but only used on the client side when applying network updates. On the server this remains empty.

3.10.2.5 next_network_id

```
uint32_t network_cs_t::next_network_id
```

Next network identifier to assign when the server spawns a new networked entity. Each server instance is responsible for guaranteeing uniqueness of network IDs across the lifetime of the session. Clients leave this at zero.

3.10.2.6 pending_destroy_ids

```
uint32_t* network_cs_t::pending_destroy_ids
```

Pending destroy list for server-side entity removal.

When an entity with a NetworkedEntity component is destroyed on the server, its `network_id` should be queued here via `network_cs_mark_entity_destroy()`. During the next network tick the queued IDs will be sent to all clients as part of the delta snapshot with a zero component count, signalling clients to destroy their corresponding local entity and remove the mapping. After transmitting to all peers the list is cleared.

3.10.2.7 protocol_handler

```
protocol_handler_t network_cs_t::protocol_handler
```

The protocol handler instance.

3.10.2.8 sync_acc

```
float network_cs_t::sync_acc
```

The synchronization accuracy factor.

The documentation for this struct was generated from the following file:

- include/network_cs.h

3.11 network_id_pair_t Struct Reference

Pair mapping between a global network ID and a local entity ID.

```
#include <network_map.h>
```

Public Attributes

- uint32_t **network_id**
Unique ID assigned by the server for an entity.
- entity_t **local_id**
Corresponding local entity ID in the ECS.

3.11.1 Detailed Description

Pair mapping between a global network ID and a local entity ID.

The documentation for this struct was generated from the following file:

- include/network_map.h

3.12 network_map_t Struct Reference

Dynamic mapping structure from network IDs to local entity IDs. This map grows dynamically as new network IDs are encountered. Clients rely on this to map persistent server-assigned IDs to their local ECS entities.

```
#include <network_map.h>
```

Collaboration diagram for network_map_t:

Public Attributes

- `network_id_pair_t * pairs`
Array of ID pairs.
- `size_t count`
Array of ID pairs.
- `size_t capacity`
Allocated capacity of the pairs array.

3.12.1 Detailed Description

Dynamic mapping structure from network IDs to local entity IDs. This map grows dynamically as new network IDs are encountered. Clients rely on this to map persistent server-assigned IDs to their local ECS entities.

The documentation for this struct was generated from the following file:

- `include/network_map.h`

3.13 network_packet_t Struct Reference

A full network packet structure. This struct combines the header and the data payload, providing a complete packet representation for sending and receiving.

```
#include <protocol_handler.h>
```

Collaboration diagram for `network_packet_t`:

Public Attributes

- `packet_header_t header`
- `uint8_t data [MAX_PACKET_SIZE - sizeof(packet_header_t)]`

3.13.1 Detailed Description

A full network packet structure. This struct combines the header and the data payload, providing a complete packet representation for sending and receiving.

3.13.2 Member Data Documentation

3.13.2.1 data

```
uint8_t network_packet_t::data [MAX_PACKET_SIZE - sizeof(packet_header_t)]
```

The packet payload.

3.13.2.2 header

```
packet_header_t network_packet_t::header
```

The packet header.

The documentation for this struct was generated from the following file:

- include/protocol_handler.h

3.14 networked_entity_t Struct Reference

Component representing network identity and interest groups.

```
#include <networked_entity.h>
```

Public Attributes

- uint32_t **network_id**
Globally unique ID assigned by the server.
- interest_mask_t **interest_groups**
Bitmask of interest groups.

3.14.1 Detailed Description

Component representing network identity and interest groups.

- `network_id`: Globally unique ID assigned by the server. Never reused.
- `interest_groups`: Bitmask of interest groups that determines which clients receive updates for this entity.

The documentation for this struct was generated from the following file:

- include/networked_entity.h

3.15 packet_header_t Struct Reference

The header of a network packet. Contains metadata about the packet, such as its size and type. Note on padding: The size is 2 bytes and the enum type is 4 bytes, which results in 2 bytes of padding to align the struct to an 8-byte boundary.

```
#include <protocol_handler.h>
```

Public Attributes

- uint16_t [size](#)
- packet_type_t [type](#)

3.15.1 Detailed Description

The header of a network packet. Contains metadata about the packet, such as its size and type. Note on padding: The size is 2 bytes and the enum type is 4 bytes, which results in 2 bytes of padding to align the struct to an 8-byte boundary.

3.15.2 Member Data Documentation

3.15.2.1 size

```
uint16_t packet_header_t::size
```

The total size of the packet in bytes.

3.15.2.2 type

```
packet_type_t packet_header_t::type
```

The type of the packet.

The documentation for this struct was generated from the following file:

- include/protocol_handler.h

3.16 peer_t Struct Reference

Represents a connected network peer.

```
#include <connection_manager.h>
```

Collaboration diagram for peer_t:

Public Attributes

- char [id](#) [64]
- struct sockaddr_in [addr_tcp](#)
- struct sockaddr_in [addr_udp](#)
- [net_socket_t](#) [net_sockets](#) [PROTOCOL_COUNT]
- bool [is_connected](#)
- bool [udp_ready](#)
- [interest_mask_t](#) [interest_mask](#)
Bitmask of interest groups for this peer.
- uint8_t [tcp_rx_buf](#) [MAX_PACKET_SIZE * 2]
- int [tcp_rx_len](#)

3.16.1 Detailed Description

Represents a connected network peer.

This struct contains all the necessary information to identify and communicate with a single remote peer, including its addresses and sockets.

3.16.2 Member Data Documentation

3.16.2.1 addr_tcp

```
struct sockaddr_in peer_t::addr_tcp
```

The TCP address of the peer.

3.16.2.2 addr_udp

```
struct sockaddr_in peer_t::addr_udp
```

The UDP address of the peer.

3.16.2.3 id

```
char peer_t::id[64]
```

The unique identifier for the peer (e.g., "IP:PORT").

3.16.2.4 interest_mask

```
interest_mask_t peer_t::interest_mask
```

Bitmask of interest groups for this peer.

Each bit corresponds to a group; clients will only receive updates for entities whose NetworkedEntity.interest_groups overlap with this mask. Initialized to all bits set (0xFFFFFFFF) for maximum interest.

3.16.2.5 is_connected

```
bool peer_t::is_connected
```

A flag indicating if the peer connection is currently active.

3.16.2.6 net_sockets

```
net_socket_t peer_t::net_sockets[PROTOCOL_COUNT]
```

Sockets used for communication with this peer.

3.16.2.7 udp_ready

```
bool peer_t::udp_ready
```

A flag indicating if the peer udp connection is ready.

The documentation for this struct was generated from the following file:

- include/connection_manager.h

3.17 position_t Struct Reference

Component data structure for entity position.

```
#include <ecs_builtin.h>
```

Public Attributes

- float [x](#)
- float [y](#)

3.17.1 Detailed Description

Component data structure for entity position.

3.17.2 Member Data Documentation

3.17.2.1 x

```
float position_t::x
```

X-axis position.

3.17.2.2 y

```
float position_t::y
```

Y-axis position.

The documentation for this struct was generated from the following file:

- include/ecs_builtin.h

3.18 protocol_handler_t Struct Reference

The protocol handler structure. Manages the state for incoming and outgoing network packets, providing functionality for packing, unpacking, and processing network data.

```
#include <protocol_handler.h>
```

Collaboration diagram for protocol_handler_t:

Public Attributes

- [network_packet_t out_packet](#)
- [network_packet_t in_packet](#)

3.18.1 Detailed Description

The protocol handler structure. Manages the state for incoming and outgoing network packets, providing functionality for packing, unpacking, and processing network data.

3.18.2 Member Data Documentation

3.18.2.1 in_packet

```
network_packet_t protocol_handler_t::in_packet
```

The buffer for the incoming packet.

3.18.2.2 out_packet

```
network_packet_t protocol_handler_t::out_packet
```

The buffer for the outgoing packet.

The documentation for this struct was generated from the following file:

- include/protocol_handler.h

3.19 rotation_t Struct Reference

Component data structure for entity rotation (quaternion format).

```
#include <ecs_builtin.h>
```

Public Attributes

- float [x](#)
- float [y](#)
- float [z](#)
- float [w](#)

3.19.1 Detailed Description

Component data structure for entity rotation (quaternion format).

3.19.2 Member Data Documentation

3.19.2.1 w

```
float rotation_t::w
```

W-axis rotation component (scalar part).

3.19.2.2 x

```
float rotation_t::x
```

X-axis rotation component.

3.19.2.3 y

```
float rotation_t::y
```

Y-axis rotation component.

3.19.2.4 z

```
float rotation_t::z
```

Z-axis rotation component.

The documentation for this struct was generated from the following file:

- `include/ecs_builtin.h`

3.20 transform_t Struct Reference

Component data structure for entity transformation. Combines position and rotation into a single component.

```
#include <ecs_builtin.h>
```

Collaboration diagram for transform_t:

Public Attributes

- [position_t](#) position
- [rotation_t](#) rotation

3.20.1 Detailed Description

Component data structure for entity transformation. Combines position and rotation into a single component.

3.20.2 Member Data Documentation

3.20.2.1 position

```
position_t transform_t::position
```

Position component.

3.20.2.2 rotation

```
rotation_t transform_t::rotation
```

Rotation component.

The documentation for this struct was generated from the following file:

- include/ecs_builtin.h

3.21 velocity_t Struct Reference

Component data structure for entity velocity.

```
#include <ecs_builtin.h>
```

Public Attributes

- float [x](#)
- float [y](#)

3.21.1 Detailed Description

Component data structure for entity velocity.

3.21.2 Member Data Documentation

3.21.2.1 x

```
float velocity_t::x
```

X-axis velocity.

3.21.2.2 y

```
float velocity_t::y
```

Y-axis velocity.

The documentation for this struct was generated from the following file:

- include/ecs_builtin.h

Chapter 4

File Documentation

4.1 config.h

```
00001 #pragma once
00002
00012
00016 // #define ECSNET_API
00017
00018 #if defined(_WIN32)
00019     #if defined(ECSNET_STATIC)
00020         // Estamos compilando/consumiendo la versión estática → no export/import
00021         #define ECSNET_API
00022     #elif defined(ECSNET_EXPORTS)
00023         // Estamos construyendo la DLL → exportamos símbolos
00024         #define ECSNET_API __declspec(dllexport)
00025     #else
00026         // Estamos consumiendo la DLL → importamos símbolos
00027         #define ECSNET_API __declspec(dllimport)
00028     #endif
00029 #else
00030     // En Linux/macOS normalmente no hace falta, pero si quieres podrías usar
    __attribute__((visibility("default")))
00031 #define ECSNET_API
00032 #endif
00033
00034 /* ===== ECS CONFIGURATION ===== */
00035
00045 #define INITIAL_ENTITY_CAPACITY 1024
00046
00052 #define INITIAL_COMPONENT_CAPACITY 32
00053
00054 /* -----
00055  * Compatibility macros
00056  *
00057  * The original implementation relied on compile-time constants MAX_ENTITIES
00058  * and MAX_COMPONENTS. To avoid extensive refactoring of all modules at
00059  * once, these macros are defined in terms of the initial capacities.
00060  * Network and legacy code can still refer to MAX_ENTITIES or MAX_COMPONENTS
00061  * without causing compilation errors. Dynamic growth is implemented in
00062  * the ECS core and can exceed these values; network modules should be
00063  * updated to use ecs->entity_capacity for correctness.
00064  */
00065 #ifndef MAX_ENTITIES
00066 #define MAX_ENTITIES INITIAL_ENTITY_CAPACITY
00067 #endif
00068 #ifndef MAX_COMPONENTS
00069 #define MAX_COMPONENTS INITIAL_COMPONENT_CAPACITY
00070 #endif
00071
00075 #define MAX_SYSTEMS 64
00076
00077
00078
00079 /* ===== NET CONFIGURATION ===== */
```

4.2 connection_manager.h

```
00001 #pragma once
```

```

00002
00003 #include "net_socket.h"
00004 #include <stdint.h>
00005 #include <stdbool.h>
00006 #include <winsock2.h>
00007 #include "config.h"
00008 // Include networked_entity.h to obtain the definition of interest_mask_t.
00009 // This header does not depend on connection_manager, so including it here
00010 // avoids circular dependencies and allows peers to have an interest mask.
00011 #include "networked_entity.h"
00012
00013
00014 #ifdef __cplusplus
00015 extern "C" {
00016 #endif
00017
00018
00022 #define PROTOCOL_COUNT 2
00023 #ifndef MAX_PACKET_SIZE
00024 #define MAX_PACKET_SIZE 1024
00025 #endif
00029 #define MAX_PEERS 32
00030
00034 typedef struct protocol_handler_t protocol_handler_t;
00035 typedef struct network_cs_t network_cs_t;
00036 typedef struct connection_manager_t connection_manager_t;
00037
00038
00045 typedef struct peer_t {
00046     char id[64];
00047     struct sockaddr_in addr_tcp;
00048     struct sockaddr_in addr_udp;
00049     net_socket_t net_sockets[PROTOCOL_COUNT];
00050     bool is_connected;
00051     bool udp_ready;
00052
00060     interest_mask_t interest_mask;
00061
00062     // TCP reassembly (used for framing)
00063     uint8_t tcp_rx_buf[MAX_PACKET_SIZE * 2];
00064     int tcp_rx_len;
00065 } peer_t;
00066
00067
00074 typedef struct connection_manager_t {
00075     peer_t peers[MAX_PEERS];
00076     int peer_count;
00077     net_socket_t listen_sockets[PROTOCOL_COUNT];
00078     bool is_server;
00079     void* user_data;
00080
00081     // Callbacks for different network events
00082     void (*on_receive)(void*, peer_t*, const void*, int);
00083     void (*on_connect)(void*, peer_t*);
00084     void (*on_disconnect)(void*, peer_t*);
00085 } connection_manager_t;
00086
00091 void connection_manager_init(connection_manager_t* connection_manager);
00092
00099 void connection_manager_add_listen_socket(connection_manager_t* connection_manager, net_socket_t
socket, socket_type_t socket_type);
00100
00106 void connection_manager_remove_peer(connection_manager_t* connection_manager, const char* peer_id);
00107
00116 int connection_manager_send_to_peer(connection_manager_t* connection_manager, const char* peer_id,
const void* data, int len);
00117
00126 int connection_manager_send_to_peer_udp(connection_manager_t* connection_manager, const char* peer_id,
const void* data, int len);
00127
00135 int connection_manager_broadcast(connection_manager_t* connection_manager, const void* data, int len);
00136
00144 int connection_manager_connect_to_server(connection_manager_t* connection_manager, const char* ip,
uint16_t port);
00145
00151 void connection_manager_update(connection_manager_t* connection_manager);
00152
00157 void connection_manager_destroy(connection_manager_t* connection_manager);
00158
00165 net_socket_t* connection_manager_get_listen_socket(connection_manager_t* connection_manager,
socket_type_t type);
00166
00167 uint16_t connection_manager_get_udp_local_port(connection_manager_t* connection_manager);
00168
00175 int connection_manager_set_peer_udp_remote_port_by_id(connection_manager_t* connection_manager, const
char* peer_id, uint16_t remote_udp_port);
00176

```

```

00184 peer_t* find_peer_by_addr(connection_manager_t* connection_manager, const struct sockaddr_in* addr,
socket_type_t socket_type);
00185
00195 peer_t* connection_manager_get_peer(connection_manager_t* connection_manager, const char* peer_id);
00196
00207 void connection_manager_set_peer_interest(connection_manager_t* connection_manager, const char*
peer_id, interest_mask_t mask);
00208
00209 #ifdef __cplusplus
00210 }
00211 #endif

```

4.3 ecs.h

```

00001 #ifndef ECS_H
00002 #define ECS_H
00003
00004 #pragma once
00005 #include <stdint.h>
00006 #include <stdbool.h>
00007 #include "ecs_types.h"
00008 #include "config.h"
00009 #include "ecs_builtin.h"
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014 typedef struct {
00015     component_t component_id;
00016     const void *data;
00017 } dirty_component_t;
00018
00026 ECSNET_API ecs_t* ecs_create();
00031 ECSNET_API void ecs_destroy(ecs_t* ecs);
00036 ECSNET_API void ecs_init(ecs_t *ecs);
00037
00043 ECSNET_API entity_t ecs_create_entity(ecs_t *ecs);
00050 ECSNET_API entity_t ecs_try_create_entity_by_id(ecs_t* ecs, entity_t id);
00051
00057 ECSNET_API void ecs_destroy_entity(ecs_t *ecs, entity_t entity);
00058
00068 ECSNET_API bool ecs_serialize_entity(ecs_t *ecs, entity_t entity, uint8_t *out_buffer, size_t
*out_size, size_t max_out_size);
00069
00076 ECSNET_API entity_t ecs_deserialize_entity(ecs_t *ecs, const uint8_t *in_buffer);
00077
00086 ECSNET_API bool ecs_add_component(ecs_t *ecs, entity_t entity, component_t component, void *data);
00087
00095 ECSNET_API void *ecs_get_component(ecs_t *ecs, entity_t entity, component_t component);
00096
00103 ECSNET_API const char *ecs_get_component_name(ecs_t *ecs, component_t component);
00104
00112 ECSNET_API bool ecs_has_component(ecs_t *ecs, entity_t entity, component_t component);
00113
00121 ECSNET_API bool ecs_is_component_dirty(ecs_t* ecs, entity_t entity, component_t component);
00122
00130 ECSNET_API bool ecs_remove_component(ecs_t *ecs, entity_t entity, component_t component);
00131
00139 ECSNET_API void ecs_mark_component_dirty(ecs_t *ecs, entity_t entity, component_t component);
00145 void ecs_set_dirty_hook(void (*hook)(entity_t));
00153 ECSNET_API int ecs_get_dirty_components(ecs_t *ecs, entity_t entity, dirty_component_t
*out_dirty_components);
00154
00161 ECSNET_API void ecs_clear_component_dirty(ecs_t *ecs, entity_t entity, component_t component);
00162
00169 ECSNET_API component_t ecs_register_component(ecs_t *ecs, component_descriptor_t
component_descriptor);
00170
00176 ECSNET_API void ecs_register_system(ecs_t *ecs, system_func_t func);
00177
00183 ECSNET_API void ecs_update(ecs_t *ecs, float dt);
00184
00185
00186 // Default components.
00187 ECSNET_API extern component_t COMPONENT_POSITION;
00188 ECSNET_API extern component_t COMPONENT_ROTATION;
00189 ECSNET_API extern component_t COMPONENT_TRANSFORM;
00190 ECSNET_API extern component_t COMPONENT_VELOCITY;
00191
00192
00197 ECSNET_API void ecs_register_builtin_systems(ecs_t *ecs);
00198
00203 ECSNET_API void ecs_register_builtin_components(ecs_t *ecs);

```

```

00204
00205
00206
00207 #ifdef __cplusplus
00208 }
00209 #endif
00210
00211 #endif

```

4.4 ecs_builtin.h

```

00001 #ifndef ECS_BUILTIN_H
00002 #define ECS_BUILTIN_H
00003
00004 #include <stdint.h>
00005 #include "ecs_types.h"
00006
00007
00008 #ifdef __cplusplus
00009 extern "C" {
00010 #endif
00011
00012 typedef struct {
00013     float x;
00014     float y;
00015 } position_t;
00016
00017 typedef struct {
00018     float x;
00019     float y;
00020     float z;
00021     float w;
00022 } rotation_t;
00023
00024 typedef struct {
00025     position_t position;
00026     rotation_t rotation;
00027 } transform_t;
00028
00029 typedef struct {
00030     float x;
00031     float y;
00032 } velocity_t;
00033
00034 // Forward declaration of the NetworkedEntity component ID (defined in ecs_builtin.c).
00035 ECSNET_API extern component_t COMPONENT_NETWORKED_ENTITY;
00036
00037 ECSNET_API void system_movement(ecs_t *ecs, float dt);
00038
00039 ECSNET_API void serialize_position(const void *data, uint8_t *out);
00040
00041 ECSNET_API void deserialize_position(const uint8_t *in, void *data);
00042
00043 ECSNET_API void serialize_rotation(const void *data, uint8_t *out);
00044
00045 ECSNET_API void deserialize_rotation(const uint8_t *in, void *data);
00046
00047 ECSNET_API void serialize_transform(const void *data, uint8_t *out);
00048
00049 ECSNET_API void deserialize_transform(const uint8_t *in, void *data);
00050
00051 ECSNET_API void serialize_velocity(const void *data, uint8_t *out);
00052
00053 ECSNET_API void deserialize_velocity(const uint8_t *in, void *data);
00054
00055
00056 #ifdef __cplusplus
00057 }
00058 #endif
00059 #endif

```

4.5 ecs_internal.h

```

00001 #ifndef ECS_INTERNAL_H
00002 #define ECS_INTERNAL_H
00003
00004 #include "ecs_types.h"
00005 #ifdef __cplusplus

```



```

00006 extern "C" {
00007 #endif
00013 typedef struct {
00014     bool in_use;
00015 } entity_meta_t;
00016
00026 struct component_storage_t {
00027     component_descriptor_t descriptor;
00028     void *data;
00029     bool *used;
00030     bool *is_dirty;
00031     size_t capacity;
00032 };
00033
00044 struct ecs_t {
00045     entity_meta_t *entities;
00046     component_signature_t *signatures;
00047     size_t entity_capacity;
00048     component_storage_t *components;
00049     size_t component_capacity;
00050     uint32_t registered_entities_count;
00051     uint32_t registered_component_count;
00052     system_func_t systems[MAX_SYSTEMS];
00053     int system_count;
00054 };
00055
00063 static void ecs_run_systems(ecs_t *ecs, float dt);
00064
00065 #ifdef __cplusplus
00066 }
00067 #endif
00068 #endif
00069
00070 #endif

```

4.6 ecs_types.h

```

00001 #ifndef ECS_TYPES_H
00002 #define ECS_TYPES_H
00003
00004 #include <stdint.h>
00005 #include "config.h"
00006 #ifdef __cplusplus
00007 extern "C" {
00008 #endif
00013 typedef struct ecs_t ecs_t;
00014
00020 typedef uint32_t entity_t;
00021
00027 typedef uint32_t component_t;
00028
00038 typedef uint64_t component_signature_t;
00039
00048 typedef void (*system_func_t)(ecs_t *ecs, float dt);
00049
00058 typedef void (*serialize_func_t)(const void *data_in, uint8_t *buffer_out);
00059
00068 typedef void (*deserialize_func_t)(const uint8_t *buffer_in, void *data_out);
00069
00076 typedef struct component_descriptor_t component_descriptor_t;
00077 typedef struct component_descriptor_t {
00078     size_t size;
00079     const char *name;
00080     serialize_func_t serialize;
00081     deserialize_func_t deserialize;
00082 } component_descriptor_t;
00083
00089 typedef struct component_storage_t component_storage_t;
00090
00091 #ifdef __cplusplus
00092 }
00093 #endif
00094
00095 #endif
00096 #endif

```

4.7 ecsnet.h

```

00001 //

```

```

00002 // Created by mesom on 16/08/2025.
00003 //
00004
00005 #pragma once
00006
00007 #include "ecs.h"
00008 #include "network_cs.h"
00009 #include "network_architecture.h"
00010 #include "ecs_builtin.h"
00011

```

4.8 net_socket.h

```

00001 #pragma once
00002 #include <stdint.h>
00003 #include <stdbool.h>
00004
00005 #include "config.h"
00006 #ifdef _WIN32
00007 #include <winsock2.h>
00008 #include <ws2tcpip.h>
00009 #else
00010 #include <sys/socket.h>
00011 #include <netinet/in.h>
00012 #include <arpa/inet.h>
00013 #endif
00014 #ifdef __cplusplus
00015 extern "C" {
00016 #endif
00020 typedef enum
00021 {
00022     SOCKET_TYPE_TCP,
00023     SOCKET_TYPE_UDP,
00024     PROTOCOL_COUNT
00025 } socket_type_t;
00026
00032 typedef struct
00033 {
00034     struct sockaddr_in addr;
00035     socket_type_t type;
00036     int fd;
00037 } net_socket_t;
00038
00044 net_socket_t net_socket_create(socket_type_t type);
00045
00051 int net_socket_set_non_blocking(net_socket_t* socket);
00052
00060 int net_socket_connect(net_socket_t* socket, char* ip, uint16_t port);
00061
00069 int net_socket_send(net_socket_t* socket, const void* data, int len);
00070
00079 int net_socket_sendto(net_socket_t* socket, const void* data, int len, const struct sockaddr_in*
    addr);
00080
00088 int net_socket_receive(net_socket_t* socket, void* buffer, int max_len);
00089
00098 int net_socket_receive_from(net_socket_t* socket, void* buffer, int max_len, struct sockaddr_in*
    sender_addr);
00099
00107 int net_socket_bind(net_socket_t* socket, const char* ip, uint16_t port);
00108
00119 uint16_t net_socket_get_local_port(const net_socket_t* s);
00126 int net_socket_listen(net_socket_t* socket, int backlog);
00127
00133 int net_socket_close(net_socket_t* socket);
00134
00141 ECSNET_API void net_socket_init(void);
00142
00149 ECSNET_API void net_socket_cleanup(void);
00150
00151 #ifdef __cplusplus
00152 }
00153 #endif
00154

```

4.9 network_architecture.h

```

00001 #pragma once

```

```

00002 #include "connection_manager.h"
00003 #include "ecs.h"
00004 #include <stdint.h>
00005 #include <stdbool.h>
00006 #ifdef __cplusplus
00007 extern "C" {
00008 #endif
00009 // Forward declarations
00010 typedef peer_t peer_t;
00011 typedef void (*on_receive_func_t)(const struct sockaddr_in *sender_addr, const void *data, int len);
00012
00016 typedef enum {
00017     ARCH_CLIENT_SERVER,
00018     ARCH_P2P,
00019 } network_architecture_type_t;
00020
00024 typedef struct {
00025     network_architecture_type_t type;
00026     const char *ip_address;
00027     uint16_t port;
00028     bool is_server;
00029     uint16_t tcp_port;
00030     uint16_t udp_port;
00031
00032     float ecs_sync_hz;
00033
00034     // Unified callbacks
00035     void (*on_peer_connected)(void* user_data, peer_t* peer);
00036     void (*on_peer_disconnected)(void* user_data, peer_t* peer);
00037     void (*on_packet_received)(void* user_data, peer_t* peer, const void* data, int len);
00038     void (*on_client_input)(void* user_data, peer_t* from, entity_t entity_id, uint8_t cmd, const
00039 void* extra, uint16_t extra_len);
00039 void* user_data;
00040 } network_architecture_config_t;
00041
00047 typedef struct network_architecture_t {
00048     network_architecture_type_t type;
00049     network_architecture_config_t config;
00050     void *impl; // A pointer to the specific network implementation (e.g., network_cs_t).
00051     ecs_t *ecs; // A pointer to the ECS instance.
00052 } network_architecture_t;
00053
00060 ECSNET_API void network_architecture_init(network_architecture_t* architecture, const
network_architecture_config_t* config, ecs_t* ecs);
00061
00067 ECSNET_API void network_architecture_update(network_architecture_t* architecture, float dt);
00068
00073 ECSNET_API void network_architecture_destroy(network_architecture_t* architecture);
00074
00082 ECSNET_API bool network_architecture_connect_to_server(network_architecture_t* architecture, const
char* ip_address, uint16_t port);
00083
00092 ECSNET_API bool network_architecture_send_to_peer(network_architecture_t* architecture, uint32_t
peer_id, const void* data, int len);
00093
00103 ECSNET_API bool network_architecture_send_entity_update(network_architecture_t* architecture, uint32_t
peer_id, entity_t entity_id, const void* component_data, int data_len);
00104
00112 ECSNET_API bool network_architecture_broadcast(network_architecture_t* architecture, const void* data,
int len);
00113
00119 ECSNET_API int network_architecture_get_peer_count(network_architecture_t* architecture);
00120
00127 ECSNET_API peer_t* network_architecture_get_peer(network_architecture_t* architecture, uint32_t
peer_id);
00128
00134 ECSNET_API connection_manager_t* network_architecture_get_connection_manager(network_architecture_t*
architecture);
00135
00136 #ifdef __cplusplus
00137 }
00138 #endif
00139

```

4.10 network_cs.h

```

00001 #pragma once
00002
00003 #include "network_architecture.h"
00004 #include "connection_manager.h"
00005 #include "protocol_handler.h"
00006 #include "ecs.h"
00007 #include "network_map.h"

```

```

00008 #include "networked_entity.h"
00009 #ifdef __cplusplus
00010 extern "C" {
00011 #endif
00012 typedef struct network_cs_t {
00013     ecs_t* ecs;
00020     network_architecture_config_t config;
00021     connection_manager_t connection_manager;
00022     protocol_handler_t protocol_handler;
00023     float sync_acc;
00030     network_map_t network_map;
00031
00038     uint32_t next_network_id;
00039
00051     uint32_t *pending_destroy_ids;
00052     size_t pending_destroy_count;
00053     size_t pending_destroy_capacity;
00054 } network_cs_t;
00055
00069 static inline void network_cs_assign_network_id(network_cs_t* cs, entity_t e, interest_mask_t
interest_groups) {
00070     if (!cs || !cs->ecs) return;
00071     // Only the server generates network IDs. The client leaves next_network_id at 0.
00072     uint32_t nid = cs->next_network_id++;
00073     networked_entity_t ne = { .network_id = nid, .interest_groups = interest_groups };
00074     ecs_add_component(cs->ecs, e, COMPONENT_NETWORKED_ENTITY, &ne);
00075 }
00076
00087 static inline void network_cs_set_peer_interest(network_cs_t* cs, const char* peer_id, interest_mask_t
mask) {
00088     if (!cs) return;
00089     connection_manager_set_peer_interest(&cs->connection_manager, peer_id, mask);
00090 }
00091
00092 // callbacks
00093
00101 void on_packet_received_cs(void* user_data, peer_t* peer, const void* data, int len);
00102
00108 void on_peer_connected_cs(void* user_data, peer_t* peer);
00109
00115 void on_peer_disconnected_cs(void* user_data, peer_t* peer);
00116
00123 ECSNET_API network_cs_t* network_cs_init(const network_architecture_config_t* config, ecs_t* ecs);
00124
00131 ECSNET_API void network_cs_update(network_cs_t* network_cs, float dt);
00132
00137 ECSNET_API void network_cs_destroy(network_cs_t* network_cs);
00138
00150 ECSNET_API void network_cs_mark_entity_destroy(network_cs_t* cs, entity_t entity);
00151
00160 ECSNET_API void network_cs_mark_network_id_destroy(network_cs_t* cs, uint32_t network_id);
00161
00162 #ifdef __cplusplus
00163 }
00164 #endif
00165

```

4.11 network_map.h

```

00001 // network_map.h
00002 #pragma once
00003
00004 #include <stdint.h>
00005 #include <stdlib.h>
00006 #include "ecs.h"
00007
00011 typedef struct {
00012     uint32_t network_id;
00013     entity_t local_id;
00014 } network_id_pair_t;
00015
00022 typedef struct {
00023     network_id_pair_t *pairs;
00024     size_t count;
00025     size_t capacity;
00026 } network_map_t;
00027
00032 static inline void network_map_init(network_map_t *map) {
00033     map->pairs = NULL;
00034     map->count = 0;
00035     map->capacity = 0;
00036 }
00037

```

```

00042 static inline void network_map_destroy(network_map_t *map) {
00043     if (map && map->pairs) {
00044         free(map->pairs);
00045         map->pairs = NULL;
00046         map->count = 0;
00047         map->capacity = 0;
00048     }
00049 }
00050
00051
00052 static inline entity_t network_map_lookup(network_map_t *map, uint32_t network_id) {
00053     if (!map || !map->pairs) return (entity_t)-1;
00054     for (size_t i = 0; i < map->count; ++i) {
00055         if (map->pairs[i].network_id == network_id) {
00056             return map->pairs[i].local_id;
00057         }
00058     }
00059     return (entity_t)-1;
00060 }
00061
00062 static inline void network_map_insert(network_map_t *map, uint32_t network_id, entity_t local_id) {
00063     if (!map) return;
00064     if (map->count >= map->capacity) {
00065         size_t new_cap = map->capacity == 0 ? 16 : map->capacity * 2;
00066         network_id_pair_t *new_pairs = (network_id_pair_t *)realloc(map->pairs, new_cap *
00067             sizeof(network_id_pair_t));
00068         if (!new_pairs) return;
00069         map->pairs = new_pairs;
00070         map->capacity = new_cap;
00071     }
00072     map->pairs[map->count].network_id = network_id;
00073     map->pairs[map->count].local_id = local_id;
00074     map->count++;
00075 }
00076
00077 static inline int network_map_remove(network_map_t *map, uint32_t network_id) {
00078     if (!map || !map->pairs) return 0;
00079     for (size_t i = 0; i < map->count; ++i) {
00080         if (map->pairs[i].network_id == network_id) {
00081             // Move the last element into this position and shrink the count
00082             map->pairs[i] = map->pairs[map->count - 1];
00083             map->count--;
00084             return 1;
00085         }
00086     }
00087     return 0;
00088 }
00089
00090
00091 static inline int network_map_remove(network_map_t *map, uint32_t network_id) {
00092     if (!map || !map->pairs) return 0;
00093     for (size_t i = 0; i < map->count; ++i) {
00094         if (map->pairs[i].network_id == network_id) {
00095             // Move the last element into this position and shrink the count
00096             map->pairs[i] = map->pairs[map->count - 1];
00097             map->count--;
00098             return 1;
00099         }
00100     }
00101     return 0;
00102 }
00103
00104
00105 static inline int network_map_remove(network_map_t *map, uint32_t network_id) {
00106     if (!map || !map->pairs) return 0;
00107     for (size_t i = 0; i < map->count; ++i) {
00108         if (map->pairs[i].network_id == network_id) {
00109             // Move the last element into this position and shrink the count
00110             map->pairs[i] = map->pairs[map->count - 1];
00111             map->count--;
00112             return 1;
00113         }
00114     }
00115     return 0;
00116 }
00117

```

4.12 include/networked_entity.h File Reference

Defines the NetworkedEntity component used for network replication.

```

#include <stdint.h>
#include <string.h>
#include "ecs.h"

```

Include dependency graph for networked_entity.h:

4.13 networked_entity.h

[Go to the documentation of this file.](#)

```

00001 // networked_entity.h
00002 #pragma once
00003
00004 #include <stdint.h>
00005 #include <string.h>
00006 #include "ecs.h"
00007
00008
00009 typedef uint32_t interest_mask_t;
00010 typedef struct {
00011     uint32_t network_id;
00012     interest_mask_t interest_groups;
00013 }
00014

```

```

00034 } networked_entity_t;
00035
00040 ECSNET_API extern component_t COMPONENT_NETWORKED_ENTITY;
00041
00051 static inline void networked_entity_serialize(const void *src, uint8_t *dst) {
00052     const networked_entity_t *ne = (const networked_entity_t *)src;
00053     memcpy(dst, &ne->network_id, sizeof(uint32_t));
00054     memcpy(dst + sizeof(uint32_t), &ne->interest_groups, sizeof(uint32_t));
00055 }
00056
00065 static inline void networked_entity_deserialize(const uint8_t *buffer_in, void *data_out) {
00066     networked_entity_t *ne = (networked_entity_t *)data_out;
00067     memcpy(&ne->network_id, buffer_in, sizeof(uint32_t));
00068     memcpy(&ne->interest_groups, buffer_in + sizeof(uint32_t), sizeof(uint32_t));
00069 }
00070
00077 static inline void ecs_register_networked_component(ecs_t *ecs) {
00078     component_descriptor_t desc = {
00079         .size = sizeof(networked_entity_t),
00080         .name = "NetworkedEntity",
00081         .serialize = networked_entity_serialize,
00082         .deserialize = networked_entity_deserialize,
00083     };
00084     COMPONENT_NETWORKED_ENTITY = ecs_register_component(ecs, desc);
00085 }

```

4.14 protocol_handler.h

```

00001 #pragma once
00002
00003 #include <stdint.h>
00004 #include "ecs.h"
00005 #include "ecs_types.h"
00006 #include "network_architecture.h"
00007 #include "connection_manager.h"
00008 #ifndef __cplusplus
00009 extern "C" {
00010 #endif
00016 #define MAX_PACKET_SIZE 1024
00017 #define INPUT_UP 0x01
00018 #define INPUT_DOWN 0x02
00019 #define INPUT_SPAWN 0x80
00020
00026 typedef enum {
00027     PACKET_TYPE_INVALID,
00028     PACKET_TYPE_ENTITY_UPDATE,
00029     PACKET_TYPE_MULTI_ENTITY_UPDATE,
00030     PACKET_TYPE_CLIENT_REGISTER,
00031     PACKET_TYPE_SERVER_ACK,
00032     PACKET_TYPE_CLIENT_INPUT,
00033 } packet_type_t;
00034
00041 typedef struct {
00042     uint16_t size;
00043     packet_type_t type;
00044 } packet_header_t;
00045
00051 typedef struct {
00052     packet_header_t header;
00053     uint8_t data[MAX_PACKET_SIZE - sizeof(packet_header_t)];
00054 } network_packet_t;
00055
00056
00062 typedef struct protocol_handler_t {
00063     network_packet_t out_packet;
00064     network_packet_t in_packet;
00065 } protocol_handler_t;
00066
00071 ECSNET_API void protocol_handler_init(protocol_handler_t *handler);
00072
00080 ECSNET_API void protocol_handler_pack_entity_update(protocol_handler_t *handler, entity_t entity_id,
00081     const uint8_t *data,
00082     uint16_t data_len);
00088 void protocol_handler_pack_client_register(protocol_handler_t *handler, uint16_t udp_port);
00089
00101 bool protocol_handler_unpack_client_register(const network_packet_t* pkt, uint16_t* out_port);
00102
00107 void protocol_handler_pack_server_ack(protocol_handler_t *handler);
00117 ECSNET_API void protocol_handler_pack_client_input(protocol_handler_t* handler, entity_t entity_id,
00118     uint8_t input_cmd, const void* extra, uint16_t extra_len);
00119

```

```
00137 ECSNET_API bool protocol_handler_unpack_client_input(const network_packet_t* pkt, entity_t* out_eid,
00138   uint8_t* out_cmd, const void** out_extra, uint16_t* out_extra_len);
00139
00140 void protocol_handler_process_received_data(protocol_handler_t *handler, ecs_t* ecs, peer_t *peer,
00141   const void *data, int len);
00142
00143 ECSNET_API void protocol_handler_send_packet(connection_manager_t *cm, const char *peer_id,
00144   protocol_handler_t *handler);
00145
00146
00147
00148 #ifdef __cplusplus
00149 }
00150 #endif
```

